

Basic syntax- input, output, comments and indentation.

Python is a simple and easy to read language. Its syntax is designed to be clean and readable.

① Comments in Python

Comments are used to explain the code. Python uses “#” for single line comments.

This is a single line comment

This line will be ignored by Python interpreter

Comments make the code easy to understand.

② Input in Python

We use input() function to take input from the user.

```
name = input("Enter your name:")  
age = input("Enter your age:")
```

input() always returns data as a string.

③ Output in Python

We use print() function to display output.

```
print("Hello, World!")  
print("My name is", name)  
print("I am", age, "years old.")
```

print() can display text, variables or both.

④ Indentation in Python

Indentation means giving spaces at the beginning of a line. Python uses indentation to define blocks of code.

```
if age >= 18:  
    print("You are an adult.")  
else:  
    print("You are a minor.")
```

Important Points :

- Use same number of spaces in a block.
- Usually 4 spaces are used for indentation.

Summary :

#	→	Used for comments.
input()	→	Used to take input from user.
print()	→	Used to display output.
Indentation	→	Used to define blocks of code.

Remember : Python is case sensitive and indentation is very important. 😊

Basic syntax - Input, Output in Python in Details

Python is a simple, easy to read and high level programming language. Every Python program follows some basic syntax rules. The two most important parts of any program are **Input** (taking data) and **Output** (displaying result).

1. BASIC SYNTAX OF PYTHON

- Python is case sensitive (name, Name and NAME are different).
- Statements are written in a new line.
- Indentation (spaces) is used to define blocks of code.
- Comments are used to explain code.
- A program starts execution from the first line.

Example :

```
# This is my first Python program
print("Hello, World!") # Output statement
print("Welcome to Python")
```

Output :

```
Hello, World!
Welcome to Python
```

2. INPUT IN PYTHON

Input is used to take data from the user at runtime. Python provides the `input()` function for this purpose.

- Syntax : `variable = input("message")`
- By default, `input()` takes data as a string. We can convert it into `int`, `float` etc. if required.

Example 1 : Taking a string input

```
name = input("Enter your name : ")
print("Hello, ", name)
```

Sample Run :

```
Enter your name : Rahul
Hello, Rahul
```

Example 2 : Taking an integer input

```
age = int(input("Enter your age : "))
print("You are", age, "years old.")
```

Sample Run :

```
Enter your age : 20
You are 20 years old.
```

Example 3 : Taking multiple inputs

```
a = int(input("Enter first number : "))
b = int(input("Enter second number : "))
sum = a + b
print("Sum = ", sum)
```

Sample Run :

```
Enter first number : 12
Enter second number : 25
Sum = 37
```

3. OUTPUT IN PYTHON

Output is used to display data on the screen. We use `print()` function.

- Syntax : `print(value1, value2, ..., sep=' ', end='\n')`
- `sep` : separator between values (default is space)
- `end` : end character (default is new line)

Example 1 : Basic output

```
print("Python is easy to learn.")
```

Output :

```
Python is easy to learn.
```

Example 2 : Output multiple values

```
name = "Rahul"
marks = 95
print("Name : ", name, "Marks : ", marks)
```

Output :

```
Name : Rahul Marks : 95
```

Example 3 : Using `sep` and `-end`

```
print("A", end=" - ")
print("B", end=" - ")
print("C") # default end is newline
```

Output :

```
A - B - C
```

- ★ Summary :
- `input()` is used to take input from the user.
 - `print()` is used to display output.
 - `input()` returns string, so use `int()`, `float()` to convert if needed.
 - Proper syntax and indentation make code correct and readable.



Basic Syntax - Comments and Indentation in Python

Comments and indentation are very important in Python. They help in understanding the code and defining the structure of a program.

1. COMMENTS IN PYTHON

Comments are used to explain the code. Python ignores comments while executing the program.

Types of Comments :

(A) Single Line Comment

Starts with # symbol. Everything after # on the same line is treated as a comment.

Example :

```
# This is a single line comment
x = 10      # x stores the value 10
y = 20      # y stores the value 20
print(x + y) # prints the sum
```

Output :

30

Comments are not executed. They are only for humans to understand the code.

(B) Multi Line Comment

Python does not have a special syntax for multi line comments.

We can use triple quotes (''' or ''') for this purpose.

Example :

```
"""
This is a
multi line comment.
It can span across
multiple lines.
"""
a = 5
b = 15
print(a * b)
```

Output :

75

Note :

- Use comments to describe what the code does.
- Comments make the program easier to read and maintain.
- Good practice : write meaningful comments.

2. INDENTATION IN PYTHON

Indentation means giving spaces at the beginning of a line.

Python uses indentation to define blocks of code. It is **mandatory**.

Example 1 : Correct Indentation

```
if x > 10:
    print("x is greater than 10")
else:
    print("x is 10 or less")
```

Output (if x = 15) :

x is greater than 10

Explanation :

- The two print() statements are indented under if and else.
- They are part of their respective blocks.

Example 2 : Wrong Indentation (will give IndentationError)

```
if x > 10:
    print("x is greater than 10")
else:
    print("x is 10 or less")
```

Error :

IndentationError:
expected an indented block

Explanation :

- print() under if is not indented.
- Python does not understand the structure.

Important Points about Indentation :

- Use consistent indentation throughout the program.
- Python recommends 4 spaces per indentation level.
- Use spaces, not tabs (mixing may cause errors).
- Indentation shows the relationship between statements.
- It helps Python know which statements belong to which block.

Example : Nested Indentation

```
num = 8
if num > 0:
    print("Positive number")
    if num % 2 == 0:
        print("Even number")
    else:
        print("Odd number")
```

Output :

Positive number
Even number

★ Good comments explain the code. Proper indentation makes the code work !



Types of Error in Python

When we write a program, errors may occur. Errors are the problems in a program that stop the normal execution of the program.

Python errors are mainly of two types :

1. Syntax Errors
(Detected by Python interpreter)

2. Exceptions
(Detected during the execution of program)

1. Syntax Errors :

- These errors occur when we violate the rules of Python syntax.
- These errors are detected by the Python interpreter during compile time.
- The program will not run if there is a syntax error.

Example :

Code	Error	Reason
<code>print("Hello World")</code>	<code>SyntaxError: invalid syntax</code>	Missing closing bracket)
<code>x = 10 if x > 5 print(x)</code>	<code>SyntaxError: expected ':'</code>	Missing colon : after if condition
<code>print "Hello"</code>	<code>SyntaxError: Missing parentheses in call to 'print'</code>	In Python 3, <code>print()</code> is a function.

2. Exceptions (Runtime Errors):

- These errors occur while the program is running.
- They are also called Runtime Errors.
- The program compiles successfully but stops during execution due to exception.

Important Types of Exceptions :

S.No.	Exception	Description	Example Code	Error Message
1.	ZeroDivisionError	Raised when we try to divide a number by zero.	<code>a = 10 b = 0 c = a / b</code>	<code>ZeroDivisionError: division by zero</code>
2.	NameError	Raised when a variable name is not found.	<code>print(x)</code>	<code>NameError: name 'x' is not defined</code>
3.	TypeError	Raised when an operation is applied to wrong data type.	<code>a = 10 print(a + "5")</code>	<code>TypeError: unsupported operand type(s) for +: 'int' and 'str'</code>
4.	ValueError	Raised when a function gets an argument of correct type but invalid value.	<code>int("abc")</code>	<code>ValueError: invalid literal for int() with base 10: 'abc'</code>
5.	IndexError	Raised when index of a sequence is out of range.	<code>lst = [1, 2, 3] print(lst[5])</code>	<code>IndexError: list index out of range</code>
6.	KeyError	Raised when a dictionary key is not found.	<code>d = {'a': 1, 'b': 2} print(d['c'])</code>	<code>KeyError: 'c'</code>
7.	FileNotFoundError	Raised when a file is not found.	<code>f = open("data.txt")</code>	<code>FileNotFoundError: [Errno 2] No such file or directory: 'data.txt'</code>

Note : ★ Syntax Errors are shown before the program runs.
★ Exceptions occur during the execution of the program.

Always handle exceptions using try-except block to prevent program crash.

Variables in Python

A variable is a name given to a memory location where data is stored. It allows us to store, retrieve and manipulate data in a program. Python is a dynamically typed language, which means we do not need to declare the type of variable explicitly.

1. Declaring Variables

- In Python, a variable is created the moment you assign a value to it.
- No need to specify the data type.
- Example :

```
x = 10          # integer
name = "Rahul"  # string
price = 99.50   # float
is_pass = True  # boolean
```

2. Rules for Variable Names

- Must start with a letter (a-z, A-Z) or underscore (_).
- Can contain letters, digits (0-9) and underscore (_).
- Cannot start with a digit.
- Cannot use special symbols like !, @, #, \$, % etc.
- Variable names are case sensitive.
- Cannot use Python keywords as variable names.

Valid Variable Names	Invalid Variable Names
age	1age (starts with digit)
Age_1	total-marks (contains -)
_value	%value (contains special symbol)
totalMarks	for (keyword)
x123	

3. Types of Variables (Built-in Data Types)

Type	Description	Example
int	Integer (whole numbers)	a = 25
float	Decimal or real numbers	b = 12.75
str	String (sequence of characters)	c = "Hello"
bool	Boolean (True or False)	d = True
list	Ordered collection (mutable)	e = [1, 2, 3]
tuple	Ordered collection (immutable)	f = (1, 2, 3)
dict	Key-value pairs	g = {"id": 1, "name": "Amit"}

4. Multiple Assignment

We can assign values to multiple variables in a single line.

```
a, b, c = 1, 2, 3
x = y = z = 0
```

5. Reassignment

- We can change the value of a variable at any time.
- Example :

```
x = 10
print(x)    # 10
x = 25
print(x)    # 25
```

6. Dynamic Typing

In Python, the type of a variable is determined by the value assigned to it, and it can change dynamically.

```
x = 5          # x is an integer
x = "Python"   # now x is a string
x = 3.14       # now x is a float
```

We do not need to declare type. Python handles it automatically.

7. Using Variables

(i) Using in Expressions

```
a = 10
b = 20
sum = a + b
print("Sum = ", sum) # Sum = 30
```

(ii) Taking Input and Storing in Variables

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Name: ", name)
print("Age: ", age)
```

Note :

Variables make a program easy to read, modify and maintain by giving meaningful names to data.



★ **Summary :** A variable in Python is a named storage location used to hold data. It is created by assigning a value and does not require explicit declaration.

Basic Data Types in Python

Python has several built-in data types. The most commonly used basic data types are : **Numeric types**, **String type**, **Boolean type** etc.

1. **NUMERIC TYPES**

Numeric types are used to store numeric values.

Type	Description	Example	Code	Output
int	Stores positive or negative whole numbers (no decimal)	10, -25, 0	a = 10 type(a)	10 <class 'int'>
float	Stores real numbers (decimal values)	3.14, -0.75, 2.0	b = 3.14 type(b)	3.14 <class 'float'>
complex	Stores complex numbers (a + bj)	3+4j, -2j	c = 3+4j type(c)	(3+4j) <class 'complex'>

Example :

```
x = 10          # integer
y = -7          # integer
z = 2.56        # float
w = -3.5        # float
p = 5 + 2j      # complex
```

Note :

- int has no decimal part.
- float has decimal part.
- complex number is written as a + bj where a and b are real numbers and j is the imaginary part.

2. **STRING TYPE (str)**

String is a sequence of characters enclosed in single quotes (''), double quotes (") or triple quotes (''' ''') or (""" """).

Example :

```
s1 = 'Hello'
s2 = "Python Programming"
s3 = '''This is
      a multi-line
      string'''
```

Operations on Strings :

- Concatenation : 'abc' + 'def' → 'abcdef'
- Repetition : 'ha' * 3 → 'hahaha'
- Indexing : s = 'Python', s[0] → 'P'
- Slicing : s[1:4] → 'yth'
- Length : len(s) → 6

String is Immutable

Once created, we cannot change individual characters of a string.

Example :

```
s = 'Hello'
s[0] = 'h' # Error
```

3. **BOOLEAN TYPE (bool)**

Boolean type has only two values : True or False. It is mainly used in conditions and logical operations.

Example :

```
a = True
b = False
type(a) # <class 'bool'>
type(b) # <class 'bool'>
```

Boolean Operations :

- and : True and False → False
- or : True or False → True
- not : not True → False
not False → True

Comparison Result (Boolean)

```
5 > 3      → True
5 == 3     → False
7 <= 7     → True
```

★ **Summary :**

- Numeric types : int, float, complex
- String type : str
- Boolean type : bool (True / False)

These basic data types are the building blocks of any Python program.

